

Revisiting Predicate Logic

LN chapters 3,4

STV 2025/26

Testing vs verification

Verification: here it means proving that a program will **always** behave correctly, as opposed to testing that only proves that some executions are correct.

- ▶ Verification is however undecidable (it cannot be generically automated).
- ▶ There are tools to assist us constructing proofs, or to automatically verify special cases (e.g. if the program has finite number of states) → outside our scope.
- ▶ More on such automation in Master courses e.g. Program Semantics & Verification.

Our learning goals

- ▶ To introduce you to **Hoare logic**: fundamental for imperative languages.
- ▶ To learn formulating your correctness arguments **formally**.
- ▶ To introduce you to some of the more advanced aspects such as the treatment of loops and program calls. This is useful for your later research, e.g. if you want to prove the correctness of your algorithm, or when your research involves development of a verification tool.

Overall plan

- ▶ Revisiting Predicate Logic, also introducing the notation we are going to use.
- ▶ Basic Hoare Logic
- ▶ More on proving the correctness of loops.
- ▶ Reasoning about more advanced language constructs, e.g. program calls, exceptions, OO.

How formal ?

- ▶ A proof is (really) *formal* if can be checked by a computer.
- ▶ Informal proof $\rightarrow$ can't be checked by a computer, easier to read by a human, but may be ambiguous.
- ▶ In this course we will train with *more* formal proofs: still human-readable, more *precise* than informal proof, but not machine formal.

Elements of our verification approach

- ▶ A simple programming language *uPL*
- ▶ Specifications are written in formulas of (1st order) predicate logic.
- ▶ Hoare logic to reduce program + specification to verification conditions (formulas in predicate logic).
- ▶ Use predicate logic to prove the verification conditions.

Overview

- ▶ Formulas
- ▶ Quantification
- ▶ Inference rules
- ▶ Proof
- ▶ Proofs involving quantification
- ▶ Some basic proof techniques:
 - ▶ Contradiction
 - ▶ Equational
 - ▶ Case split
 - ▶ Induction

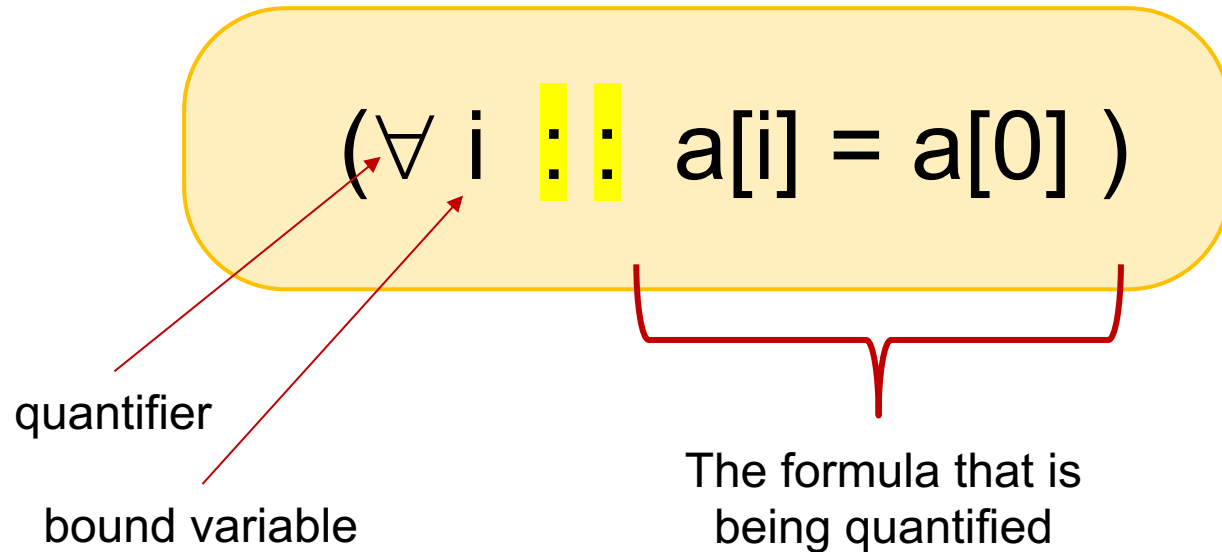
Our Formula-language (LN Ch.2)

- ▶ Language elements:
 - ▶ Variables e.g. x, y, z ...
 - ▶ Constants e.g. `true`, `false`, `0`, `1` ...
 - ▶ Arrays e.g. `a[i]`
 - ▶ Operators e.g. $+$, $-$, $<$, $=$, $\wedge$, $\Rightarrow$...
 - ▶ Quantifiers $\forall$ $\exists$,
- ▶ Types: `bool`, `int`, arrays of primitives. We usually leave types implicit in the formulas.
- ▶ Arrays are assumed to have infinite size.

**From now on write your
formulas, incl pre/post-
condition in the predicate logic
notation.**

Don't use in-code notation
(to avoid confusing yourself)

Quantified formula, basic form



It means: “for **all** i , $a[i] = a[0]$ holds.” Implicitly i is of type `int`, as it is being used as an index of an array.

Quantified formula with domain

$$(\forall i : 0 \leq i < n : a[i] = a[0])$$

domain part :
a formula to
restrict the
domain of the
quantification

range part :
the formula
that is being
quantified

It means: “for **all** i such that $0 \leq i < n$, $a[i] = a[0]$ holds.” Again, implicitly i is an int, as it is being used as an index of an array.

Domain part in quantified formula

- ▶ Definition of the form with domain part:

$$\begin{aligned}\text{▶ } (\forall x : \mathbf{P\ x} : Q\ x) &= (\forall x :: \mathbf{P\ x} \Rightarrow Q\ x) \\ \text{▶ } (\exists x : \mathbf{P\ x} : Q\ x) &= (\exists x :: \mathbf{P\ x} \wedge Q\ x)\end{aligned}$$

- ▶ Notice that the def. above implies :
 - ▶ $(\forall i : \mathbf{true} : a[i] > 0) = (\forall i :: a[i] > 0)$
 - ▶ $(\exists i : \mathbf{true} : a[i] > 0) = (\exists i :: a[i] > 0)$

Quantifying over “empty domain”

- ▶ Quantifying over “false” is also called quantifying over “empty domain”. Their meaning:
- ▶ $(\forall x : \text{false} : Q\ x)$
= $(\forall x :: \text{false} \Rightarrow Q\ x)$
= $(\forall x :: \text{true})$
= true
- ▶ $(\exists x : \text{false} : Q\ x)$
= $(\exists x :: \text{false} \wedge Q\ x)$
= $(\exists x :: \text{false})$
= false

Scoping and Nesting

- ▶ A quantifier has a “scope” :

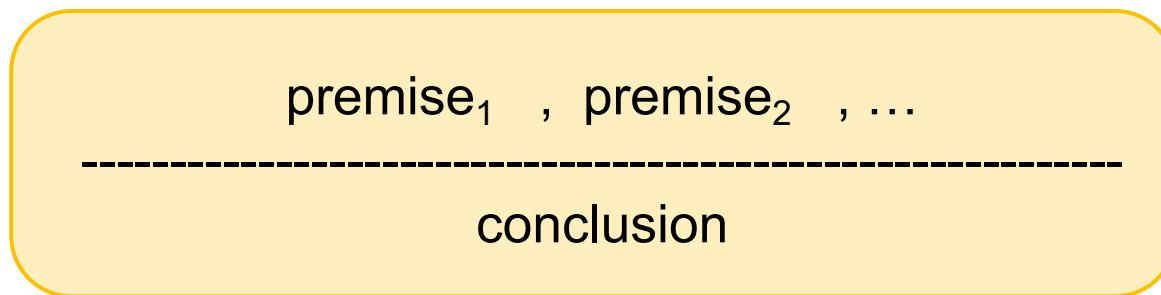
$$(\exists i : i > 0 : b[i]) \wedge \neg b[i]$$

- ▶ “**Bounded** variable” e.g. blue i in the quantified formula above.
- ▶ “**Free** variable” e.g. b and the yellow i on the right in the above example.
- ▶ Quantification can also be “nested” :

$$(\forall i :: (\exists j :: a[j] > a[i]))$$

How do we prove our claims ?

- ▶ In logic we use **inference/proof rules**. Such a rule is usually shown in this form:



- ▶ A **proof** is essentially a series of invocations of inference rules, that produces our claim from known facts and the given assumptions.

Some examples of inference rules

► Modus Ponens

$$\frac{P \quad , \quad P \Rightarrow Q}{Q}$$

► $\forall$ elimination ($\forall$ instantiation)

$$\frac{P a \quad , \quad (\forall x : P x : Q x)}{Q a}$$

Proof format (LN Ch.3)

- ▶ Stick to the proof format as in the LN (so that we have certainty when evaluating your work).
- ▶ To improve training, we deliberately make the format more explicit/verbose.
- ▶ The format will allow you to mix a **deductive style** and an **equational style** of reasoning in one proof.
 - ▶ Deductive reasoning: one way (from assumptions to conclusion)
 - ▶ Equational reasoning: bi-directional.
- ▶ Many of our example proofs will be “quite trivial”, but remember that our goal is to train you in formal reasoning (of the correctness of your program). So your “mental process” is just as important.

Basic elements of our proof format

PROOF main

[A1:] $(\forall i : i > 0 : a[i] = i)$

[A2:] $x > 10$

[G:] $a[x] > 10$

If successful this will prove the claim: $A1 \wedge A2 \Rightarrow G$ (note the implication)

BEGIN

1. { follows from A2 } $x > 0$
2. { $\forall$ -elim on A1 using 1 } $a[x] = x$
3. { rewrite A2 with 2 } $a[x] > 10$

END

Subproof and proof scope

PROOF main

...

4. ...

5. { see subproof below } $0 \leq k \Rightarrow b[k]$

PROOF sub

[A:] $0 \leq k$

[G:] $b[k]$

...

... $k+1 > 0$

...

Suppose in the subproof you manage to derive $k+1 > 0$. Using this for further inference **within** the same subproof is ok.

7. { because “sub” says $k+1 > 0$ } $k+2 > 0$



Using facts inferred in a subproof as an argument in a higher level proof is not sound!

Introducing and eliminating quant.

► Eliminating $\forall$:

$$\frac{P a \quad , \quad (\forall x : P x : Q x)}{Q a}$$

► Introducing $\exists$:

$$\frac{P a \quad , \quad Q a}{(\exists x : P x : Q x)}$$

► How about introducing $\forall$ and eliminating $\exists$??

Introducing $\forall$

PROOF main

[A:] $(\forall k: k \geq 0 : b[k])$

[G:] $(\forall k: k > 1: b[k])$

1. { how ?? } $(\forall k: k > 1 : b[k])$

PROOF sub

[A:] $k > 1$

[G] $b[k]$

1. { follows from A } $k \geq 0$

2. { $\forall$ -elim on main.A using 1 } $b[k]$

Success! (conclusion: $k > 1 \Rightarrow b[k]$)

We then could argue that since k is **unconstrained** in the parent proof, we can generalize the normal conclusion of this subproof to $(\forall k : k > 1 : b[k])$. But this is a bit error prone.

$\forall$ Introduction through a subproof

PROOF main

[A:] $(\forall k: k \geq 0 : b[k])$

[G:] $(\forall k: k > 1: b[k])$

1. { see subproof } $(\forall k: k > 1: b[k])$

PROOF sub

ANY k

[A:] $k > 1$

[G] $b[k]$

1. { follows from A } $k \geq 0$

2. { $\forall$ -elim on main.A using 1 } $b[k]$

ANY k marker at the start of a proof introduces a scope for k , namely limited within the proof. Within the proof, occurrences of k refers to this k and any **previous** assumption about this k **cannot** be used. From such a proof we are allowed to conclude a **generalized** formula, in this case $(\forall k: k > 1: b[k])$

Proof by Contradiction

- ▶ Let's prove this claim:

$$\begin{aligned} & a[i] > i \quad \wedge \quad (\exists i : 0 \leq i : a[i] = i) \\ & \Rightarrow \\ & \neg(\forall i : 0 \leq i : a[i] > i) \end{aligned}$$

- ▶ We'll prove this *by contradiction*. Based on this rule:

$$\frac{\neg Q \Rightarrow \text{false}}{Q}$$

Top level proof

PROOF main

[A1:] $a[i] > i$

[A2:] $(\exists i : 0 \leq i : a[i] = i)$

[G:] $\neg (\forall i : 0 \leq i : a[i] > i)$

BEGIN

1. { see subproof } $(\forall i : 0 \leq i : a[i] > i) \Rightarrow \text{false}$
2. { rule of contradiction on 1 } $\neg (\forall i : 0 \leq i : a[i] > i)$

END

Eliminating $\exists$

PROOF main

[A1:] $a[i] > i$

[A2:] $(\exists i : 0 \leq i : a[i] = i)$

[G:] $\neg(\forall i : 0 \leq i : a[i] > i)$

BEGIN

Eliminating $\exists$ introduces a **[SOME k]** marker which imposes a scope. From this point on, occurrences of k refers to this k and any **previous** assumption about this k **cannot** be used.

PROOF sub

[A:] $(\forall i : 0 \leq i : a[i] > i)$

[G:] false

1. $\{\exists$ **elimination** on main.A2} **[SOME k]** $0 \leq k \wedge a[k] = k$
2. $\{\forall$ elimination on A using 1st conjunct of 1 $\} a[k] > k$
3. $\{\text{contradiction of 1 and 2}\}$ false

END

....

Equational proof

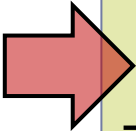
EQUATIONAL PROOF

[A:] $0 \leq n$

[D:] $\text{found } n = (\exists i : 0 \leq i < n : b[i])$

This equational proof proves

$0 \leq n \Rightarrow (\text{found } (n+1) = \text{found } n \wedge b[n])$,
with the given definition of “found n”.



$\text{found } (n + 1)$

$= \{ \text{def. found} \}$

$(\exists i : 0 \leq i < n+1 : b[i])$

$= \{ \text{Domain Merge Thm A.4.16 justified by A} \}$

$(\exists i : 0 \leq i < n \vee i=n : b[i])$

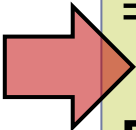
$= \{ \text{Domain Split Thm A.4.12} \}$

$(\exists i : 0 \leq i < n : b[i]) \vee (\exists i : i=n : b[i])$

$= \{ \text{Quantification over Singleton Thm A.4.10} \}$

$(\exists i : 0 \leq i < n : b[i]) \vee b[n]$

$= \{ \text{def. found} \}$



$\text{found } n \vee b[n]$

END

Proof with case split

- ▶ Suppose that to prove Q , we identify N -cases, and we want to prove Q separately for each case.
- ▶ Based on this inference rule:

$$\frac{P_1 \vee P_2, \quad P_1 \Rightarrow Q, \quad P_2 \Rightarrow Q}{Q}$$

- ▶ Example, prove this:

$$b[n] \wedge (\forall i : i < n : b[i]) \Rightarrow (\forall i : i < n+1 : b[i])$$

Top level proof of the example

PROOF main

[A1:] $b[n]$

[A2:] $(\forall i : i < n : b[i])$

[G:] $(\forall i : i < n+1 : b[i])$

BEGIN

1. { see the proof below } G

PROOF sub

ANY i

[A:] $i < n+1$

[G:] $b[i]$

BEGIN

1 { follows from A }

2 { see subproof sub_1 }

3 { see subproof sub_2 }

4 { Case Split on 1,2,3 }

END

$i < n \vee i = n$

$i < n \Rightarrow b[i]$

$i = n \Rightarrow b[i]$

$b[i]$

END

One more example of $\forall$ -intro and $\exists$ -elimination

PROOF

[G:] $(\forall n: n \bmod 4 = 0 : n \bmod 2 = 0)$

BEGIN

1. { see the proof below }

PROOF

ANY n

[A1:] $n \bmod 4 = 0$

[G:] $n \bmod 2 = 0$

BEGIN

1. {def. of mod on A1} $(\exists k:: n = 4*k)$

2. { $\exists$ -elim on 1}. [SOME k] $n = 4*k$

3. {arith. on 2} $n = 2*2k$

4. { $\exists$ -intro on 3} $(\exists m:: n = 2*m)$

5. {def. mod on 4} $n \bmod 2 = 0$

END

END

for $a > 0$:

$$p \bmod a = 0 \Leftrightarrow (\exists k:: p = a*k)$$

Proof with induction

- ▶ Induction over “natural numbers” is based on this rule:

$$\frac{P\ 0 \quad , \quad (\forall n: 0 \leq n : P\ n \Rightarrow P\ (n+1))}{(\forall n: 0 \leq n : P\ n)}$$

(n is implicitly assumed to be of type int)

Example of a proof with induction

PROOF

[G:] $(\forall n: 0 \leq n: (n^3 + 2n) \bmod 3 = 0)$

BEGIN

1. { trivial } $(0^3 + 2*0) \bmod 3 = 0$
2. { see below } $(\forall n: 0 \leq n: (n^3 + 2n) \bmod 3 = 0 \Rightarrow ((n+1)^3 + 2(n+1)) \bmod 3 = 0)$

PROOF Ind

ANY n

[A1:] $0 \leq n$

[A2:] $(n^3 + 2n) \bmod 3 = 0$

[G:] $((n+1)^3 + 2(n+1)) \bmod 3 = 0$

BEGIN

1. { def. **mod** on A2 } $(\exists k: n^3 + 2n = 3*k)$
2. { $\exists$ -elim } **[SOME k]** $n^3 + 2n = 3*k$
3. { see subproof }
 $(n+1)^3 + 2(n+1) = 3(k + n^2 + n + 1)$

EQUATIONAL PROOF

$$\begin{aligned}
 & (n+1)^3 + 2(n+1) \\
 &= \{\text{arithm.}\} (n^3 + 3n^2 + 3n + 1) + (2n + 2) \\
 &= \{\text{arithm.}\} (n^3 + 2n) + 3n^2 + 3n + 3 \\
 &= \{\text{Ind.2}\} 3*k + 3*(n^2 + n + 1) \\
 &= \{\text{arithm.}\} 3(k + n^2 + n + 1)
 \end{aligned}$$

4. {2, $\exists$ -intro} $(\exists m: ((n+1)^3 + 2(n+1)) = 3*m)$
 5. {def. **mod**} $((n+1)^3 + 2(n+1)) \bmod 3 = 0$
- END**

3. { induction, using 1,2 } $(\forall n: 0 \leq n: (n^3 + 2n) \bmod 3 = 0)$

END

$$p \bmod 3 = 0 \Leftrightarrow (\exists k :: p = 3*k)$$

Lists

(LN Ch.4)

Why lists ?

- ▶ Our arrays are infinite. On the other hand, we sometimes need to specify aggregate properties of finite segments of an array → we will use a list.
 - ▶ So, we add list to one Formula-language.
 - ▶ For convenience, we'll use Haskell like notations to express properties of lists.
- ▶ Proving equivalence between different implementations of list functions (to extend what you learned in the course Functional Programming)

Some list notation

- ▶ `[ ]`, `[1, 2, 3]`
- ▶ `x:s`, `s ++ t`
- ▶ Enumeration: (a and be below are integers)
 - ▶ `[a .. b]`
 - ▶ `[a .. b)`
- ▶ List comprehension, e.g.: `[ 2*i | i from s , isOdd i ]`

List functions

- ▶ On lists we can define functions like SUM, MAX, COUNT etc:

$$\begin{aligned}\mathbf{SUM} \ [] &= 0 \\ \mathbf{SUM} \ (x:s) &= x + \mathbf{SUM} \ s\end{aligned}$$

- ▶ We can map array over finite domain to list, and thus can define notions like SUM, MAX over a finite array-segment by defining them over the corresponding list.

Converting array to list

- ▶ Let a be an array (infinite), and s be a list of integers. We define:

$$a\ s = [a[i] \mid i \text{ from } s]$$

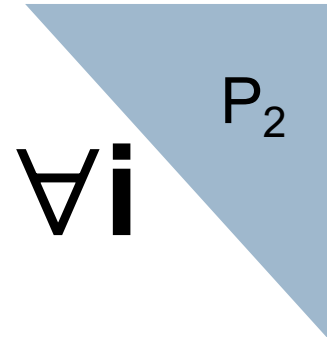
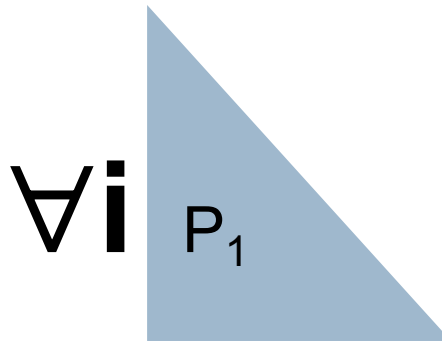
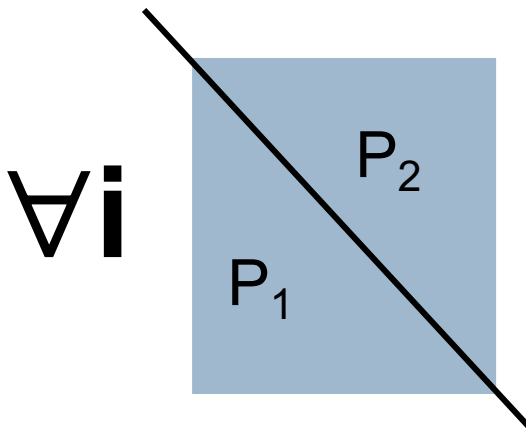
- ▶ So now formulas like these are well defined:
 - ▶ $a[0..n)$: the segment of the array a , starting from index 0 up to but not including n .
 - ▶ **SUM** ($a[0..n)$) : the sum of the elements of the segment-array $a[0..n)$.
 - ▶ Similarly: **COUNT** ($a[0..n)$) , **MAX** ($a[0..n)$) , ...

Domain split

► For $\forall, \exists$ (Thm A.4.12) :

$$(\forall i : P_1 \vee P_2 : Q i) = (\forall i : P_1 : Q i) \wedge (\forall i : P_2 : Q i)$$

$$(\exists i : P_1 \vee P_2 : Q i) = (\exists i : P_1 : Q i) \vee (\exists i : P_2 : Q i)$$



Inspire solutions for the problem of computing $\forall$ over $P_1 + P_2$ by dividing it into smaller problems.

Domain Split for other “quantifiers”

- ▶ Analogous, e.g. :

$$(\sum i : P_1 i \text{ } \mathbf{V} \text{ } P_2 i : i) = (\sum i : P_1 i : i) \text{ } \mathbf{+} \text{ } (\exists i : P_2 i : i)$$

But only if P_1 and P_2 are disjoint!

- ▶ On the other hand:

$$\mathbf{SUM} (s \text{ } \mathbf{++} \text{ } t) = \mathbf{SUM} s \text{ } \mathbf{+} \text{ } \mathbf{SUM} t$$

Now we can do SUM-split on array

EQUATIONAL PROOF

[A:] $k \geq 0$

$$\begin{aligned} & \text{SUM } (a[0 \dots k+1]) \\ &= \{ \text{enumeration split, Thm A.5.8, justified by A} \} \\ & \text{SUM } (a[0 \dots k] ++ [k]) \\ &= \{ \text{comprehension split Thm A.5.12} \} \\ & \text{SUM } (a[0 \dots k] ++ [a[k]]) \\ &= \{ \text{domain split of SUM (prev. slide ... see also Thm A.5.16)} \} \\ & \text{SUM } (a[0 \dots k]) + \text{SUM } [a[k]] \\ &= \{ \text{def. SUM} \} \\ & \text{SUM } (a[0 \dots k]) + a[k] \end{aligned}$$

END

Some standard 'splitting' thms

▶ **SUM** ($s \mathbin{++} t$) = **SUM** s + **SUM** t

▶ **COUNT** ($s \mathbin{++} t$) = **COUNT** s + **COUNT** t

▶ **MAX** ($s \mathbin{++} t$) = **MAX** s max **MAX** t

// provided s, t are non-empty

Induction

- ▶ Some standard thms in your Appendix for convenience.
- ▶ Some properties may require induction to prove. We'll use this list-induction rule:

$$\frac{(\forall x, s :: P\ s \Rightarrow P(x:s))}{(\forall s :: P\ s)}$$

- ▶ Simple example, prove:

COUNT $s \geq 0$, for all s

Top level proof

PROOF main

[G:] $(\forall s :: \text{COUNT } s \geq 0)$

BEGIN

1 { from def. COUNT } $\text{COUNT } [] \geq 0$

2 { subproof } $(\forall x, s :: \text{COUNT } s \geq 0 \Rightarrow \text{COUNT } (x:s) \geq 0)$

PROOF Pinduct

ANY x, s

[A:] $\text{COUNT } s \geq 0$

[G:] $\text{COUNT } (x:s) \geq 0$

...

...

END

3 { list-induction on 1 and 2 } $(\forall s :: \text{COUNT } s \geq 0)$

END

Reasoning about functional programs

- ▶ This is also covered in the course Functional Programming. We will illustrate it with an example.
- ▶ Example, two functional programs to do reverse a list :

$$\begin{aligned}\text{rev } [] &= [] \\ \text{rev } (x : s) &= \text{rev } s ++ [x]\end{aligned}$$
$$\begin{aligned}\text{rv } t [] &= t \\ \text{rv } t (x : s) &= \text{rv } (x : t) s\end{aligned}$$

The first one is intuitive, the 2nd is much faster. Prove that they are equivalent. More precisely, prove : $\text{rev } s = \text{rv } [] s$, for all s .

Top level proof

PROOF main

[G:] $(\forall s :: \text{rev } s = \text{rv } [] s)$

BEGIN

1 { def. of rev and rv } $\text{rev } [] = \text{rv } [] []$

2 { subproof } $(\forall x, s :: (\text{rev } s = \text{rv } [] s) \Rightarrow (\text{rev } (x:s) = \text{rv } [] (x:s)))$

3 { list-induction on 1 and 2 } G

END

Attemp-1

PROOF main

[G:] $(\forall s :: \text{rev } s = \text{rv } [] s)$

BEGIN

1 { def. of rev and rv } $\text{rev } [] = \text{rv } [] []$
2 { subproof below } $(\forall x, s :: (\text{rev } s = \text{rv } [] s) \Rightarrow (\text{rev } (x:s) = \text{rv } [] (x:s)))$

Induction hypothesis (IH)

Essentially:

$\text{rev } (x:s)$
 $= \{ \text{def. rev} \} \text{rev } s ++ [x]$
 $= \{ \text{IH} \} \text{rv } [] s ++ [x]$
 $= \{ ?? \text{ cannot find a justification for this!} \} \text{rv } [x] s$
 $= \{ \text{def. rev} \} \text{rv } [] (x : s)$

We need more information to progress. Basically a way to turn $\text{rev } s ++ t$ to $\text{rv } t s$.

3 { list-induction on 1 and 2 } G

END

Attemp-2: let's prove this first

PROOF main

[G:] $(\forall s :: (\forall t :: \text{rev } s ++ t = \text{rv } t \text{ } s))$

BEGIN

1 { subproof-1 } $(\forall t :: \text{rev } [] ++ t = \text{rv } t \text{ } [])$

2 { subproof-2 below }

$(\forall x, s :: (\forall t :: \text{rev } s ++ t = \text{rv } t \text{ } s) \Rightarrow (\forall t :: \text{rev } (x:s) ++ t = \text{rv } t \text{ } (x:s)))$

Induction hypothesis (IH)

Essentially:

$$\begin{aligned} & \text{rev } (x:s) ++ t \\ &= \{ \text{def. rev} \} (\text{rev } s ++ [x]) ++ t \\ &= \{ \text{associativity of } ++ \text{ and def. " : " } \} \text{rev } s ++ (x : t) \\ &= \{ \text{IH} \} \text{rv } (x : t) s \\ &= \{ \text{def. rev} \} \text{rv } t \text{ } (x : s) \end{aligned}$$

3 { list-induction on 1 and 2 } G

END

Back to the original problem

- ▶ The main claim to prove was $(\forall s :: \text{rev } s = \text{rv } [] s)$
- ▶ Along the way, we proved a lemma:

$$(\forall s :: (\forall t :: \text{rev } s ++ t = \text{rv } t s))$$

- ▶ But notice that this lemma **directly** implies the original claim (without having to do prove the latter through induction)., namely by instantiating t to $[]$. I leave the formal proof of this to you.